Clean CPU evidence: 0.7.0-registry-4f60380

Measured source `4f60380b4a5ac07931347b4f9eba0ac696983f9a`, tree `f8a34552b825c7f8106a05ec34d7731d6f2538e7`; dirty before/after: false. Library source: registry 0.7.0. One pinned CPU; f64 operations and explicitly named f32 constructors; row-major inputs.

Workload / library	Per-call p50 / p95 (us)	Batch-mean p50 / p95 (us)	Rust allocations	Peak additional requested bytes	Process peak RSS (MiB)
cholesky_factor_64_f64/faer	8.562 / 11.954	8.663 / 9.174	2	33280	4.21
cholesky_factor_64_f64/kernix	22.814 / 27.409	24.173 / 26.170	2	65536	4.13
cholesky_factor_64_f64/nalgebra	14.206 / 16.507	15.040 / 15.577	1	32768	3.82
cholesky_factor_64_f64/numpy	10.214 / 10.841	11.123 / 11.849	unavailable	unavailable	33.27
create_ones_64x64_f32/candle	0.232 / 0.243	0.243 / 0.253	7	16696	4.70
create_ones_64x64_f32/faer	0.179 / 0.182	0.179 / 0.182	1	16384	4.19
create_ones_64x64_f32/kernix	0.186 / 0.191	0.200 / 0.214	1	16384	4.09
create_ones_64x64_f32/ndarray	0.160 / 0.214	0.147 / 0.172	1	16384	3.83
create_ones_64x64_f32/numpy	1.332 / 1.744	1.404 / 1.578	unavailable	unavailable	32.90
create_zeros_64x64_f32/candle	0.167 / 0.168	0.160 / 0.176	5	16648	4.54
create_zeros_64x64_f32/faer	0.172 / 0.178	0.173 / 0.214	1	16384	4.15
create_zeros_64x64_f32/kernix	0.146 / 0.148	0.143 / 0.153	1	16384	3.93
create_zeros_64x64_f32/ndarray	0.099 / 0.100	0.094 / 0.099	1	16384	3.93
create_zeros_64x64_f32/numpy	0.421 / 0.470	0.431 / 0.457	unavailable	unavailable	32.84
matmul_128_f64/candle	72.785 / 80.284	76.432 / 79.892	9	131352	5.62
matmul_128_f64/faer	84.278 / 92.174	85.796 / 89.410	1	131072	4.65
matmul_128_f64/kernix	117.968 / 134.735	123.302 / 133.632	3	327680	4.58
matmul_128_f64/nalgebra	118.759 / 128.007	120.198 / 147.777	2	327680	4.23
matmul_128_f64/ndarray	118.998 / 137.571	120.820 / 130.095	2	327680	4.19
matmul_128_f64/numpy	78.228 / 86.911	80.629 / 92.434	unavailable	unavailable	34.33
matmul_32_f64/candle	1.422 / 1.921	1.649 / 1.755	9	8592	4.38
matmul_32_f64/faer	1.578 / 1.868	1.642 / 1.682	1	8192	4.26
matmul_32_f64/kernix	1.670 / 2.115	1.709 / 1.752	3	24576	4.14
matmul_32_f64/nalgebra	1.588 / 1.742	1.515 / 1.581	2	24576	3.92
matmul_32_f64/ndarray	1.473 / 1.496	1.526 / 1.598	2	24576	3.93
matmul_32_f64/numpy	2.894 / 3.360	2.698 / 2.815	unavailable	unavailable	33.07
matmul_batched_4x16x32x16_f64/candle	1.970 / 2.241	2.260 / 2.392	25	25504	5.24
matmul_batched_4x16x32x16_f64/kernix	4.124 / 4.285	2.167 / 2.258	7	16392	4.48
matmul_batched_4x16x32x16_f64/numpy	3.300 / 4.130	3.252 / 4.261	unavailable	unavailable	33.15
matmul_sum_backward_32x16x8_f64/candle	4.457 / 5.160	4.319 / 4.838	113	27988	4.86
matmul_sum_backward_32x16x8_f64/kernix	2.367 / 2.813	2.532 / 2.728	16	18376	4.37
qr_256x64_f64/faer	413.236 / 493.690	426.167 / 466.773	7	331776	4.83
qr_256x64_f64/kernix	522.773 / 598.110	516.535 / 552.400	6	557568	4.96
qr_256x64_f64/nalgebra	382.268 / 434.673	392.464 / 516.270	4	295424	4.05
qr_256x64_f64/numpy	359.863 / 474.264	366.347 / 388.259	unavailable	unavailable	34.31
qr_96x32_f64/faer	66.159 / 74.979	68.251 / 70.276	7	67584	4.73
qr_96x32_f64/kernix	43.656 / 50.906	46.127 / 47.186	6	106752	4.34
qr_96x32_f64/nalgebra	37.460 / 42.795	38.455 / 39.257	4	57600	3.94

Workload / library	Per-call p50 / p95 (us)	Batch-mean p50 / p95 (us)	Rust allocations	Peak additional requested bytes	Process peak RSS (MiB)
qr_96x32_f64/numpy	40.097 / 49.417	41.721 / 44.404	unavailable	unavailable	33.40
softmax_256x64_f64/candle	79.866 / 88.537	80.525 / 83.952	37	265200	5.72
softmax_256x64_f64/kernix	86.270 / 122.759	87.889 / 89.717	1	131072	4.42
softmax_256x64_f64/ndarray	75.605 / 85.958	75.224 / 80.577	1	131072	4.24
softmax_256x64_f64/numpy	107.298 / 118.287	105.477 / 111.471	unavailable	unavailable	33.18
training_backward_2x4x8x4_f64/candle	14.552 / 15.339	15.233 / 16.365	472	8332	6.07
training_backward_2x4x8x4_f64/kernix	9.065 / 9.266	9.769 / 10.103	71	4224	4.39
training_forward_2x4x8x4_f64/candle	3.801 / 7.613	3.690 / 5.656	126	3912	5.86
training_forward_2x4x8x4_f64/kernix	4.364 / 7.249	4.784 / 5.150	34	1040	4.38
training_record_2x4x8x4_f64/candle	5.214 / 5.930	5.541 / 9.302	149	10712	5.79
training_record_2x4x8x4_f64/kernix	4.864 / 5.033	5.392 / 5.496	39	6832	4.51
training_sgd_2x4x8x4_f64/kernix	0.481 / 0.505	0.544 / 0.562	5	688	4.49

Method and limits

- Per-call quantiles use nearest ranks over 256 individual warmed calls; batch quantiles use 15 means over separate 10-ms batches. They are separate distributions. Raw samples remain in raw.json.
- Eight warmup calls follow validation. Timing includes allocation, dispatch, computation and output destruction. The first call field excludes output destruction. Inputs and library/runtime imports are prepared outside timing.
- Three extra processes record their first operation calls. These are first-use observations, not cache-flushed or device-startup latency. Timer-pair overhead is reported without subtraction; tiny calls are clock-sensitive.
- ndarray and Candle softmax are explicit stable compositions, not claims about a native softmax API.
- Constructor cases allocate owned 64x64 f32 tensors. Pre-0.6 registry baselines use full(value), explicitly marked in metadata; current workspace runs use zeros/ones. No nested literal reinterpretation is involved.
- All GEMM inputs are C-order and match elementwise. Native output layout differs by library. QR materializes thin Q and R in every implementation. Cholesky returns reusable factors (or NumPy L); no extra factor materialization is timed.
- Kernix checks finite values/symmetry. Some competitors validate a narrower contract. No finite-data precondition check was removed to improve timing. Accuracy results and budgets are retained per worker.
- The backward case uses an actual reverse-mode graph in Kernix and Candle for sum(A @ W), with A constant and W trainable. Both copy input/weight data and returned gradients; no manual derivative is used as a framework ranking.
- Allocations are measured in a separately compiled instrumented binary after warmup, outside all timing loops. Counts cover Rust global-allocator requests, not native allocator internals, stack memory or all NumPy allocations. NumPy allocation counts are unavailable, not zero.
- Requested-byte peak is additional live Rust allocation during one call, including output; it is not scratch-only memory or peak RSS. RSS is the process high-water mark including fixtures, validation and runtime, read from Linux VmHWM. Other platforms may report unavailable.
- The published cases are allocation-inclusive CPU APIs and graph workloads. Pure internal kernel-only and end-to-end application/model latency are not measured. No workload result implies a complete application frame rate.
- faer is explicitly serial; Rayon and BLAS thread counts are one. No GPU comparison. Python/NumPy timings include Python dispatch. Backend and dependency versions/configuration, governor and RAM are recorded.
- Training stages use batched matmul + bias + layer_norm + tanh + step-two feature slice + MSE (Candle index_select). Forward excludes graph recording; record includes parameter/input/target snapshots and eager forward/loss; backward reuses a prepared graph and returns fresh derivatives. Allocation probes measure the corresponding stage, including graph values/scratch, not graph-container-only bytes. SGD uses fixed precomputed real gradients and includes transactional validation/parameter updates; it is not an end-to-end converging training loop. Candle normalization is an explicit population-statistics composition; its

checks/rounding differ. Historical registry baselines without these Tape operations omit the stages, recorded in metadata, rather than substitute manual derivatives.

- A shared laptop has uncontrolled load/frequency/cache effects. Report comparator wins and losses; no universal ranking follows. The source archive comes from committed bytes, with result directories excluded. Library dependencies are pinned by the saved driver lockfile.